Base de Datos I

Índices y optimización de consultas

Centro de e-Learning
UTN.BA

Índices



- Es una estructura que almacena datos copiados del campo de la tabla sobre el cual se definió el índice.
- Su función es permitir encontrar en forma más eficiente un registro de una tabla.
- La forma de almacenar los índices varía en cada Sistema de Gestión de Bases de Datos, pero lo importante es que todos establecen la separación en el nivel físico.

Centro de e-Learning UTN.BA

Sintaxis

```
CREATE [ UNIQUE ] INDEX [ CONCURRENTLY ] [ name ] ON table [ USING method ]
( { column | ( expression ) }[ ASC | DESC ] [ NULLS { FIRST | LAST } ] [, ...] )
[WHERE predicate]
```

- **Concurrently**: permite crear el índice sin bloquear la tabla
- **Method**: uno de btree, hash, gin, gist, spgist, brin. (Si no se especifica asume btree)
- **Expression**: permite definir una expresión para crear el índice.
- **Predicate**: expresión para definir un índice parcial.

Centro de e-Learning UTN.BA



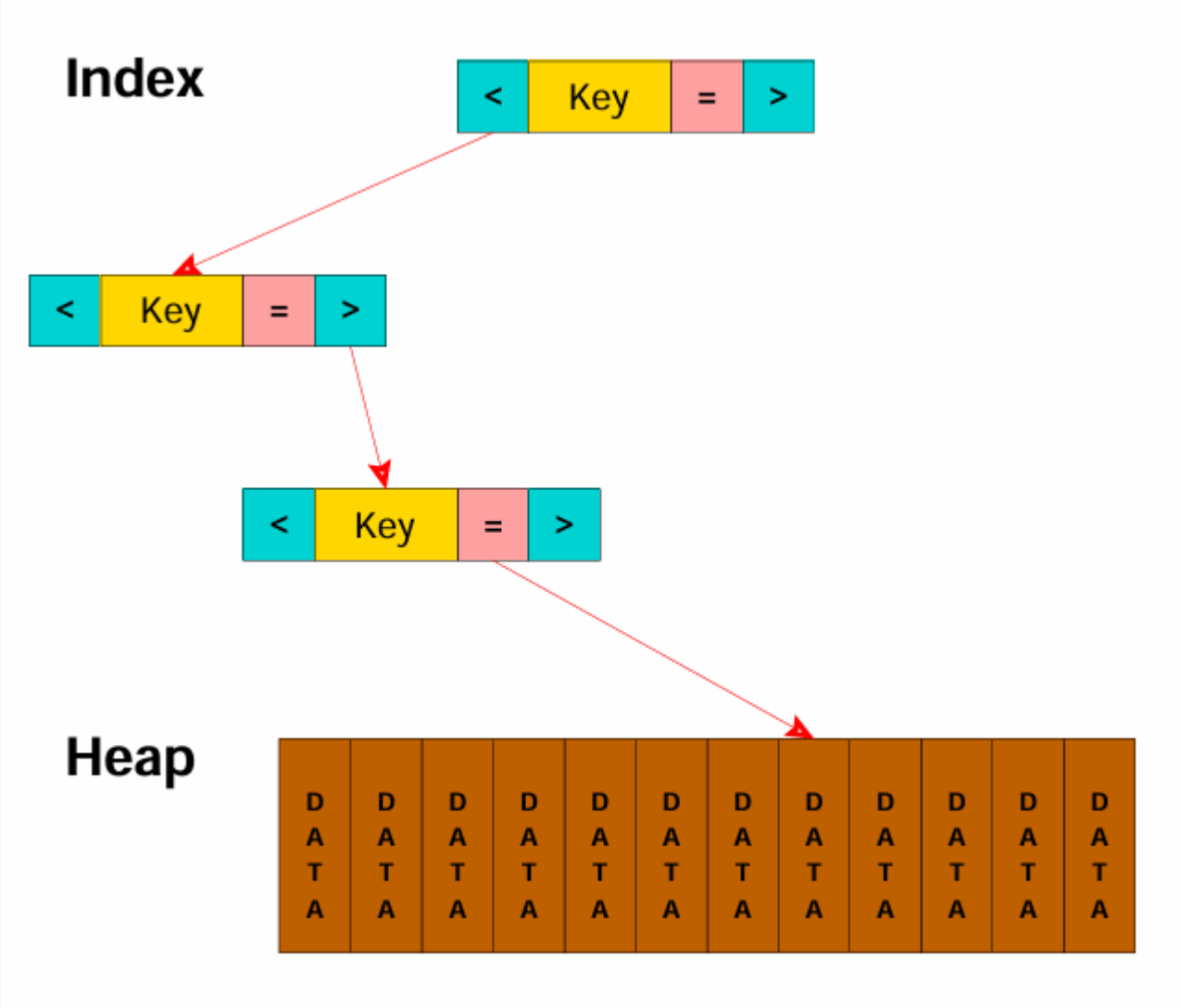
Funcionamiento de los índices

- Cuando se realiza una consulta a la base de datos, el Procesador de Consultas define si debe recorrer la tabla completa (escaneo secuencial) o si utilizará alguno de los índices definidos para obtener los registros solicitados de forma más rápida.
- Si la tabla posee pocos registros, siempre será más rápido un acceso secuencial que un acceso a través de alguno de los índices definidos.
- Cada actualización a una tabla (crear, modificar o eliminar un registro) impactará en los índices que la misma posea, por ende, una tabla con muchos índices se traducirá en una baja en el rendimiento en las operaciones de alta, baja o modificación de registros.

Centro de e-Learning UTN.BA

Índices B-Tree

- Ideal sobre columnas que posean valores únicos.
- Puede proveer los datos ordenados
- Mejor performance en temas de concurrencia



Centro de e-Learning UTN.BA

Generalized Search Tree (GIST)

- Ideal para indexar datos complejos que no se “adaptan” a un índice b-tree
- Permite indexar datos geográficos, arrays, par clave/valor

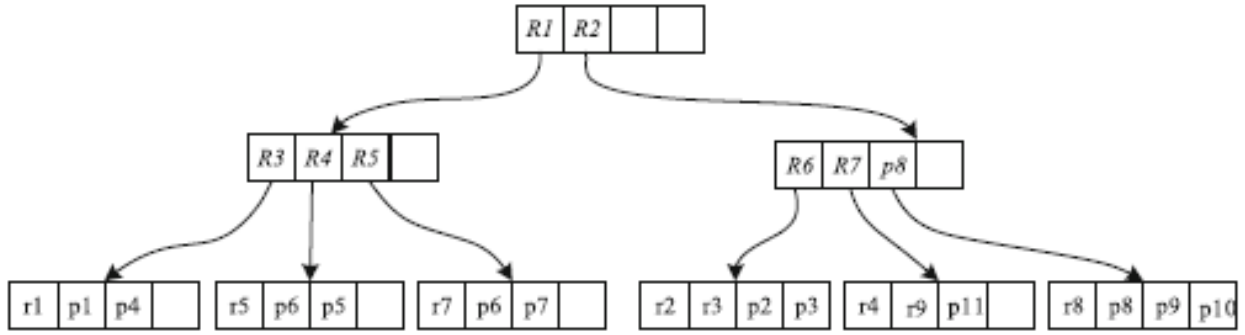
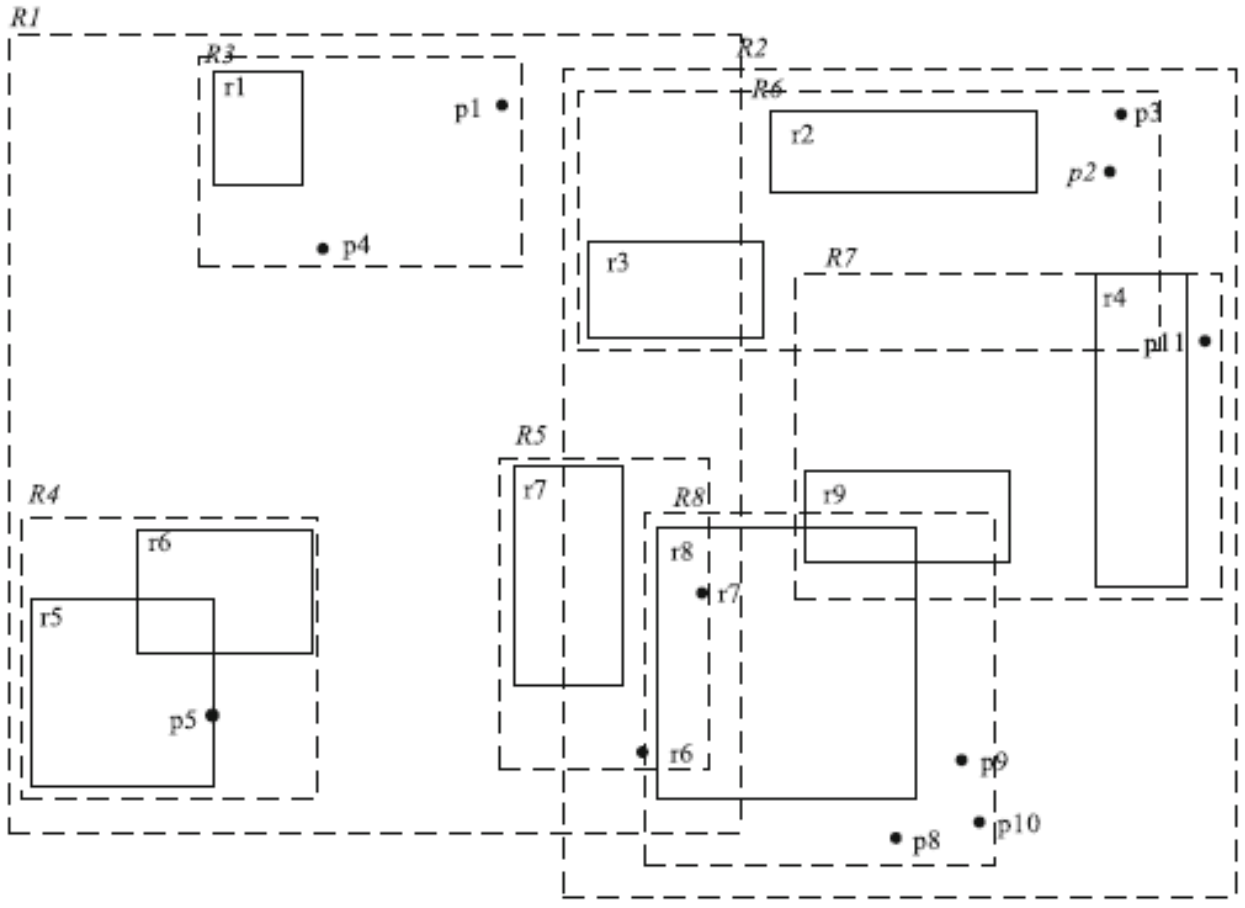


Fig. 2.7



Centro de e-Learning UTN.BA

Otros tipos de índices

Generalized Inverted Index (GIN)

- Ideal para indexar valores compuestos
- Ideal para indexar columnas con muchos duplicados, ya que por cada clave puede tener punteros a varias filas. Esto también reduce el tamaño del índice
- Este índice guarda un conjunto de pares (clave, valor) donde el "valor" es un conjunto de IDs de filas donde ocurre esa clave
- Muy utilizado en valores de tipo: documento, JSON, arrays

Otros

- Índices por expresiones (Expression Index)
  - Permite definir el índice a partir de una expresión, función, concatenación, etc.
  - Ej: lower(nombre)
- Índices parciales (Partial Index)
  - Permite definir un índice para un subconjunto de filas y no toda la tabla
  - Reduce el overhead en operaciones de insert/update
  - Reduce el tamaño del índice
  - Ej: CREATE INDEX state_idx ON customer (state) WHERE state = 'AZ';

Centro de e-Learning UTN.BA

Clustering



Clustering es reordenar los datos de una tabla de acuerdo a un índice creado previamente y reescribirla en disco.

Sintaxis

```
CLUSTER [VERBOSE] table_name [ USING index_name ]
```

Ejemplo

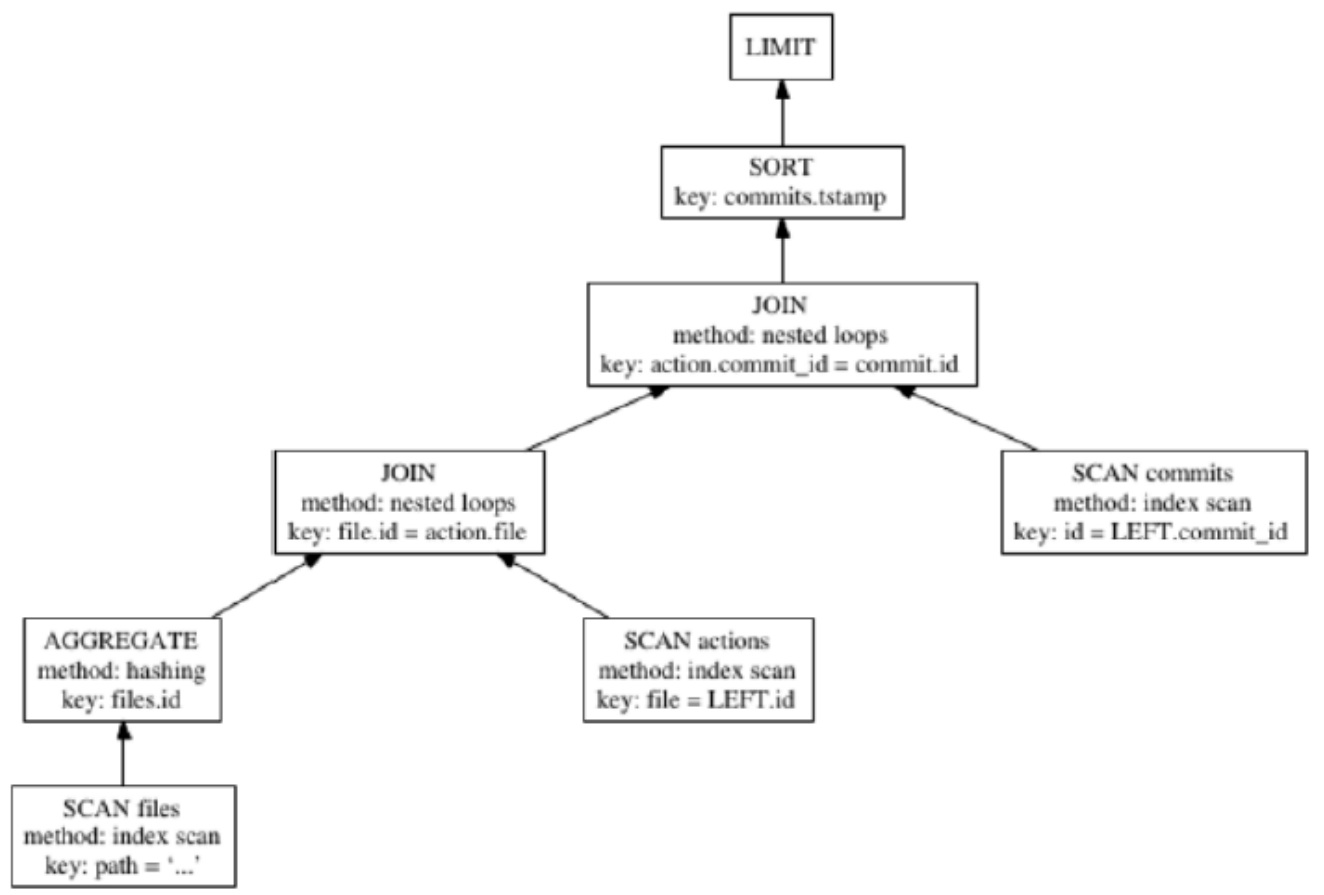
```
CREATE INDEX geom_idx ON TABLE my_tabla_geo USING GIST(the_geom);

CLUSTER my_tabla_geo USING geom_idx;
```


Centro de e-Learning UTN.BA

Optimización de consultas

- SQL es un lenguaje declarativo, esto significa que podemos decirle QUÉ hacer, pero no CÓMO.
- El motor de BD por cada consulta evalúa distintos planes de ejecución seleccionando el menos costoso.




```sql
SELECT DISTINCT
    commits.id,
    commits.tstamp,
    actions.id AS action_id,
    files.id AS file_id,
    files.path
FROM files
JOIN actions ON actions.file = files.id
JOIN commits ON actions.commit_id = commits.id
WHERE files.path = '...'
ORDER BY commits.tstamp DESC
LIMIT 50;
```


Centro de e-Learning UTN.BA

El comando EXPLAIN

El comando **EXPLAIN** permite ver, evaluar y mejorar el plan de ejecución

```
EXPLAIN select * from table_dummy;

EXPLAIN VERBOSE select * from table_dummy;
```

Por cada plan de ejecución que el motor realice se mostrará una fila (un nodo del árbol).

Para interpretar la salida del comando EXPLAIN, estos nodos deben leerse desde abajo hacia arriba.

Cada nodo presenta la siguiente información:

- Costo estimado para comenzar a devolver información
- Costo estimado para finalizar la consulta
- Cantidad estimada de filas procesadas
- Cantidad estimada de bytes

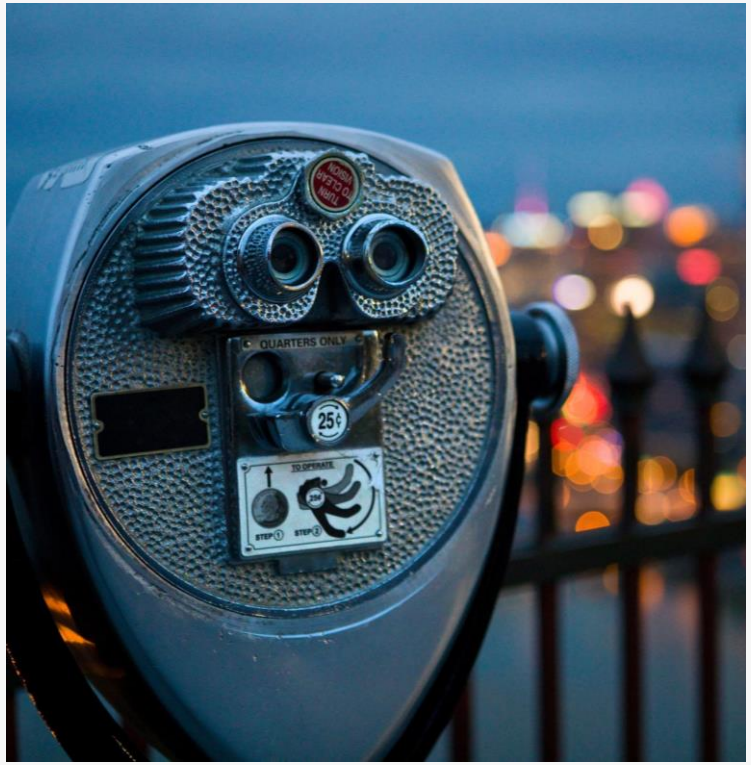
¿Qué tiene que evaluar el planner?

- Scan Methods (sequential, index, bitmap index)
- Join Methods (nested loops, hash join, merge join)
- Aggregation Methods (hash, sort)

```
QUERY PLAN
-------------------------------------------------------------
Seq Scan on table_dummy (cost=0.00..458.00 rows=10000 width=244)
```


Centro de e-Learning UTN.BA

Scan Methods



- Son los métodos que utiliza el motor de base de datos para leer los datos de las tablas. Puede ser secuencial o indexado.
- El motor necesita conocer la distribución de los datos porque en función a ello determina el método de escaneo a realizar sobre la tabla.
- No siempre un acceso indexado es más eficiente que un acceso secuencial.
- Se puede forzar el uso de índices con los siguientes comandos:
    - SET enable_seqscan = false;
    - SET enable_bitmapscan = false;
    - SET enable_nestloop = false

- El caso de que el motor siga eligiendo un plan que no involucra el uso de índices, puede deberse a causas más fundamentales, como por ejemplo que la condición del query no utilice campos del índice.

Centro de e-Learning UTN.BA

Join Methods



- Son los métodos que utiliza el motor de base de datos para unir/cruzar los datos de las tablas. Es una de las cosas más importantes a tener en cuenta en la optimización de consultas.

Nested loops

Hash join

Merge join

Centro de e-Learning UTN.BA

Nested loops



Sequential Scan

- Ocurre ante un join entre 2 tablas y lo que hace el motor es comparar todas las filas de la tabla 1 contra todas las filas de la tabla 2. Sólo es útil cuando las tablas involucradas tienen pocas filas ya que lo realiza de forma muy rápida.

Index scan

- Ocurre cuando hay creados índices sobre las tablas y se realiza un join estableciendo un filtro. En este caso, el motor va a realizar un nested loop, pero utilizando la información proporcionada por los índices. Con lo cual sólo va recorrer las filas que verifiquen la condición establecida en ambas tablas.

Centro de e-Learning UTN.BA

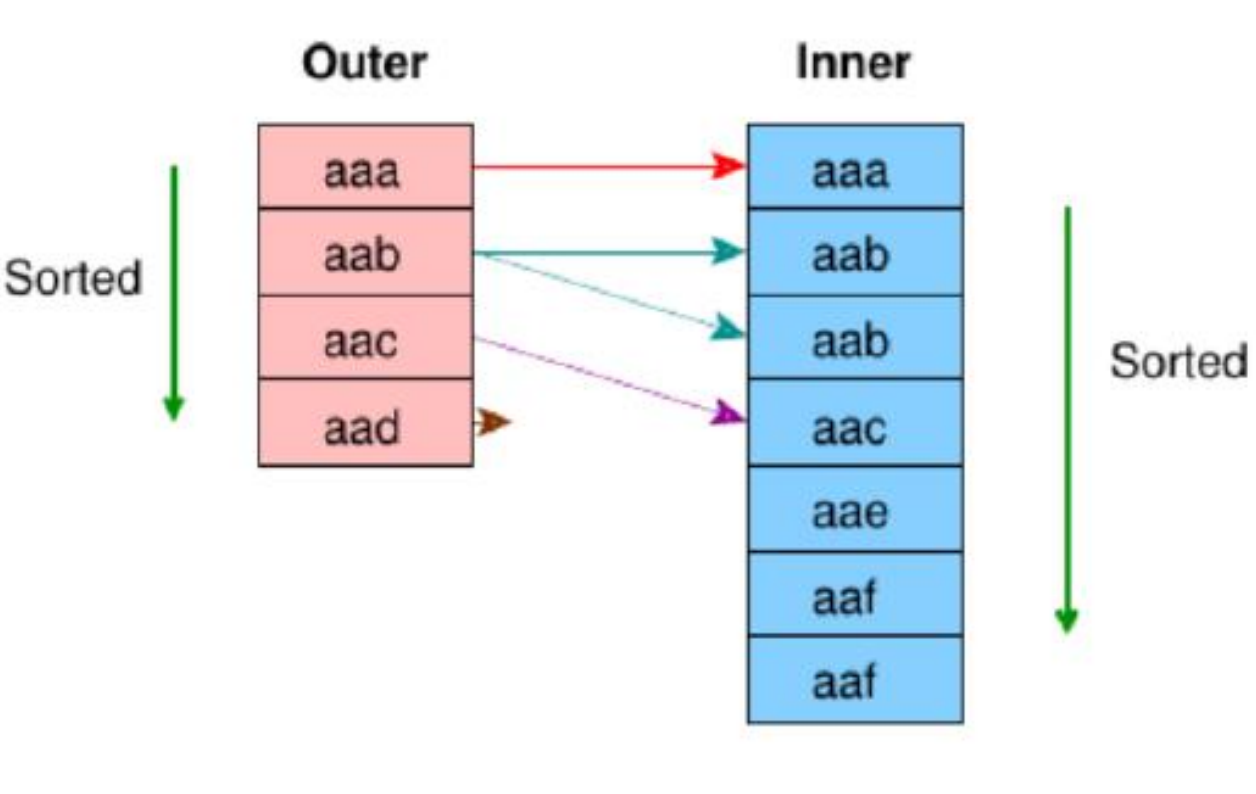
Hash Join



- Crea una estructura en memoria donde guarda el hash de las claves de una tabla y luego recorre dicha estructura efectuando comparaciones contra los valores de la otra tabla involucrada. Es uno de los joins más comunes elegidos por el motor.
- Es efectivo, siempre y cuando la tabla sobre la cual se genera el hash pueda ser almacenada en memoria.
- Por lo general ocurre cuando se establece algún tipo de restricción sobre alguna tabla del tipo mayor o menor.

Centro de e-Learning UTN.BA

Merge Join



- El motor ordena ambas tablas y luego las recorre casi en paralelo devolviendo las filas que machean la condición del join. Este tipo de join ocurre cuando se establece un join sin una restricción.
- Es útil para grandes tablas ya que no requiere que la tabla sea levantada en memoria y porque no se realizan múltiples recorridas sobre las tablas.
- Si las tablas poseen algún tipo de índice que establece un orden, entonces el motor evita ordenar las tablas previamente.

Centro de e-Learning UTN.BA

Comandos especiales



- ANALYZE: ayuda a recolectar estadísticas sobre la distribución de los datos
- VACUUM: limpia índices y tablas.

Centro de e-Learning UTN.BA

¡Gracias!

Centro de e-Learning

UTN.BA